



Universidad Veracruzana

Implementación de la app rastreo en Django

Endpoint y token de seguridad

Programación segura

Dr. Juan Manuel Gutiérrez Méndez

Propósito de la práctica

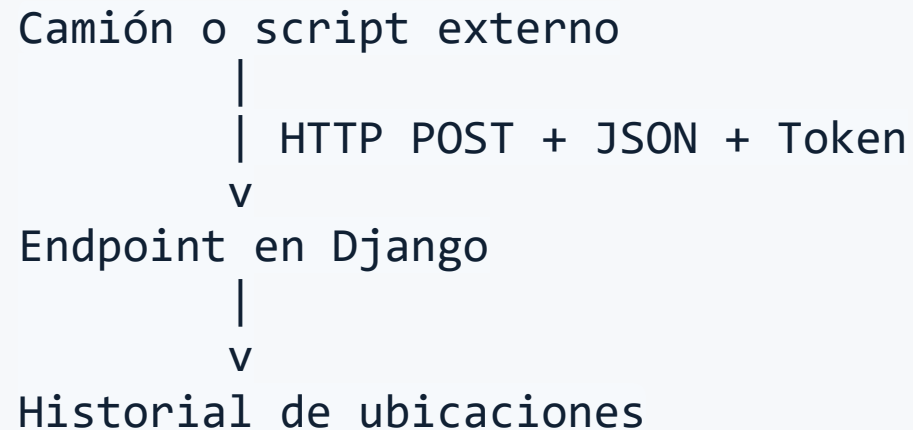
Implementar una nueva app llamada `rastreo` dentro del proyecto de logística para recibir y guardar ubicaciones de camiones mediante un endpoint protegido con token.

La práctica integra entrada por red, validación de token, validación de datos y persistencia en SQLite.

Problema a resolver

La aplicación logística necesita recibir información enviada por un camión o por un script que simule al camión.

El sistema debe guardar un historial de ubicaciones para consultar posteriormente el movimiento de las unidades.



Dos entradas al sistema

ENTRADA WEB

- Usada por clientes y supervisores.
- Requiere sesión de usuario.
- Usa roles y grupos de Django.
- Responde con páginas HTML.

ENTRADA API

- Usada por el camión o script externo.
- Requiere token.
- Recibe datos en JSON.
- Responde con JSON.

Decisión arquitectónica

La app `rastreo` será una app de dominio porque representa una funcionalidad del negocio: registrar ubicaciones.

El módulo `seguridad` se mantiene como una carpeta transversal al mismo nivel que `apps`.

``rastreo`` resuelve el caso de uso; ``seguridad`` concentra controles reutilizables.

Estructura esperada

```
logistica/  
├── apps/  
│   ├── bienes/  
│   ├── clientes/  
│   └── rastreo/  
│       ├── models.py  
│       ├── views.py  
│       ├── urls.py  
│       └── apps.py  
├── seguridad/  
│   ├── __init__.py  
│   ├── roles.py  
│   ├── decorators.py  
│   ├── tokens.py  
│   └── validaciones.py  
└── config/  
    ├── settings.py  
    └── urls.py
```

Resultado esperado

Al finalizar, el proyecto tendrá:

- una app `rastreo` ;
- un modelo `UbicacionCamion` ;
- un endpoint `/api/camiones/ubicacion/` ;
- validación de token en el módulo transversal `seguridad` ;
- validación de latitud, longitud, velocidad y fecha;
- almacenamiento del historial en SQLite;
- pruebas con `curl` o script Python.

Paso 1. Crear la app rastreo

Desde la raíz del proyecto:

```
python manage.py startapp rastreo apps/rastreo
```

Si el proyecto ya utiliza la carpeta `apps/` , la app debe quedar dentro de esa estructura.

```
apps/rastreo/  
├── __init__.py  
├── admin.py  
├── apps.py  
├── migrations/  
├── models.py  
├── tests.py  
└── views.py
```


Paso 2. Ajustar apps.py

```
# apps/rastreo/apps.py

from django.apps import AppConfig

class RastreoConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "apps.rastreo"
```

Este archivo permite que Django identifique correctamente la aplicación dentro de la carpeta `apps`.

Paso 3. Registrar la app en settings.py

```
# config/settings.py

INSTALLED_APPS = [
    # Apps de Django
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    # Apps del proyecto
    "apps.bienes",
    "apps.clientes",
    "apps.rastreo",
]
```

El módulo `seguridad` no se registra en `INSTALLED_APPS` porque no es una app Django.

Paso 4. Definir el modelo de ubicación

La app `rastreo` guardará cada posición recibida como un registro histórico.

```
# apps/rastreo/models.py

from django.db import models

class UbicacionCamion(models.Model):
    camion_id = models.CharField(max_length=30)
    latitud = models.DecimalField(max_digits=9, decimal_places=6)
    longitud = models.DecimalField(max_digits=9, decimal_places=6)
    velocidad = models.DecimalField(max_digits=6, decimal_places=2)
    registrado_en = models.DateTimeField()
    recibido_en = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.camion_id} - {self.registrado_en}"
```

Explicación del modelo

Campo	Tipo	Propósito
camion_id	Texto	Identifica la unidad que envía la ubicación
latitud	Decimal	Coordenada geográfica norte-sur
longitud	Decimal	Coordenada geográfica este-oeste
velocidad	Decimal	Velocidad reportada por la unidad
registrado_en	Fecha y hora	Momento reportado por el camión
recibido_en	Fecha y hora	Momento en que Django recibió el registro

Nota sobre el modelo

En esta práctica se guardará el identificador del camión como texto.

En una versión más avanzada podría existir un modelo `Camion` y `UbicacionCamion` tendría una llave foránea hacia ese modelo.

Para esta práctica, el objetivo es comprender el endpoint, el token, la validación y el historial de ubicaciones.

Paso 5. Crear y aplicar migraciones

```
python manage.py makemigrations rastreo  
python manage.py migrate
```

Con esto se crea la tabla correspondiente en SQLite.

Django usará el modelo `UbicacionCamion` para guardar cada ubicación válida recibida por el endpoint.

Paso 6. Crear el módulo de validaciones

Las validaciones de parámetros se colocarán en el módulo transversal `seguridad`.

```
seguridad/  
├── __init__.py  
├── roles.py  
├── decorators.py  
├── tokens.py  
└── validaciones.py
```

El archivo `validaciones.py` concentrará reglas reutilizables para revisar datos de entrada.

Validación decimal (ejemplo)

```
# seguridad/validaciones.py

from decimal import Decimal, InvalidOperation

def convertir_decimal(valor, nombre_campo):
    try:
        return Decimal(str(valor))
    except (InvalidOperation, TypeError):
        raise ValueError(f"El campo {nombre_campo} debe ser numérico.")
```

Esta función convierte valores recibidos desde JSON a `Decimal`.

Validación de latitud

```
# seguridad/validaciones.py

from decimal import Decimal

def validar_latitud(valor):
    ...

    return latitud
```

La latitud válida se encuentra entre `-90` y `90` .

Validación de longitud

```
# seguridad/validaciones.py

from decimal import Decimal

def validar_longitud(valor):
    longitud = convertir_decimal(valor, "longitud")

    ...

    return longitud
```

La longitud válida se encuentra entre `-180` y `180` .

Validación de velocidad

```
# seguridad/validaciones.py

from decimal import Decimal

def validar_velocidad(valor):
    velocidad = convertir_decimal(valor, "velocidad")

    ...

    return velocidad
```

La velocidad no debe aceptar valores negativos.

Validación de fecha y hora

```
# seguridad/validaciones.py

from django.utils.dateparse import parse_datetime

def validar_fecha_hora(valor):
    fecha = parse_datetime(str(valor))

    if fecha is None:
        raise ValueError(
            "El campo registrado_en debe tener formato ISO 8601."
        )

    return fecha
```

Ejemplo de valor esperado:

```
2026-05-24T10:30:00
```

Paso 7. Crear validación de token

El token también se colocará en `seguridad`, porque representa una regla transversal de control de acceso técnico.

```
# seguridad/tokens.py

from django.conf import settings

def obtener_token_bearer(request):
    encabezado = request.headers.get("Authorization", "")

    if not encabezado.startswith("Bearer "):
        return None

    token = encabezado.replace("Bearer ", "", 1).strip()

    if not token:
        return None

    return token
```

Paso 8. Configurar token en settings.py

```
# config/settings.py  
  
TOKEN_RASTREO_CAMIONES = "TOKEN_DE_PRACTICA_CAMIONES"
```

Para esta práctica, el token será estático.

Utiliza una clave apropiada para el token

Comentario didáctico sobre el token

- Un token estático permite introducir el concepto de autenticación técnica en servicios web.
- Sin embargo, si el token se filtra, otra persona podría enviar ubicaciones falsas.
- Esta situación debe discutirse con el grupo como una limitación del diseño.

Paso 9. Crear el endpoint en rastreo/views.py

```
# apps/rastreo/views.py

import json

from django.http import JsonResponse
from django.views.decorators.csrf import csrf_exempt
from django.views.decorators.http import require_POST

from seguridad.tokens import obtener_token_bearer, token_valido
from seguridad.validaciones import (
    validar_fecha_hora,
    validar_latitud,
    validar_longitud,
    validar_velocidad,
)

from .models import UbicacionCamion
```



```
@csrf_exempt
@require_POST
def registrar_ubicacion(request):
    token = obtener_token_bearer(request)

    if token is None or not token_valido(token):
        return JsonResponse(
            {"error": "Token inválido o ausente."},
            status=401
        )

    try:
        datos = json.loads(request.body)
    except json.JSONDecodeError:
        return JsonResponse(
            {"error": "El cuerpo debe ser JSON válido."},
            status=400
        )

    camion_id = datos.get("camion_id")
```

```

if not camion_id:
    return JsonResponse(
        {"error": "El campo camion_id es obligatorio."},
        status=400
    )

try:
    latitud = validar_latitud(datos.get("latitud"))
    longitud = validar_longitud(datos.get("longitud"))
    velocidad = validar_velocidad(datos.get("velocidad"))
    registrado_en = validar_fecha_hora(datos.get("registrado_en"))
except ValueError as error:
    return JsonResponse(
        {"error": str(error)},
        status=400
    )

ubicacion = UbicacionCamion.objects.create(
    camion_id=camion_id,
    latitud=latitud,
    longitud=longitud,
    velocidad=velocidad,
    registrado_en=registrado_en
)

return JsonResponse(
    {
        "mensaje": "Ubicación registrada correctamente.",
        "ubicacion_id": ubicacion.id
    },
    status=201
)

```

Qué hace el endpoint

Paso	Acción
1	Recibe una solicitud <code>POST</code>
2	Extrae el token del encabezado <code>Authorization</code>
3	Valida que el token sea correcto
4	Lee el cuerpo JSON
5	Verifica que exista <code>camion_id</code>
6	Valida latitud, longitud, velocidad y fecha
7	Guarda el registro en <code>UbicacionCamion</code>
8	Devuelve una respuesta JSON

Sobre csrf_exempt

El endpoint será usado por un cliente externo, no por un formulario web con sesión de navegador.

Por eso, para esta práctica se usa:

```
@csrf_exempt
```

Pero se compensa con:

- token obligatorio;
- método `POST` ;
- validación del cuerpo JSON;
- validación de datos;
- respuestas controladas.

Paso 10. Crear urls.py en rastreo

```
# apps/rastreo/urls.py

from django.urls import path
from . import views

urlpatterns = [
    path(
        "api/camiones/ubicacion/",
        views.registrar_ubicacion,
        name="registrar_ubicacion"
    ),
]
```

Esta ruta será el endpoint que consumirá el camión o el script externo.

Paso 11. Integrar rutas en config/urls.py

```
# config/urls.py

from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),

    path("", include("apps.bienes.urls")),
    path("", include("apps.clientes.urls")),
    path("", include("apps.rastreo.urls")),
]
```

Con esto, el endpoint queda disponible dentro del proyecto principal.

Paso 12. Ejecutar el servidor

```
python manage.py runserver
```

El endpoint quedará disponible en ambiente local:

```
http://127.0.0.1:8000/api/camiones/ubicacion/
```

Paso 13. Probar con curl

```
curl -X POST http://127.0.0.1:8000/api/camiones/ubicacion/ \
-H "Content-Type: application/json" \
-H "Authorization: Bearer TOKEN_DE_PRACTICA_CAMIONES" \
-d '{
  "camion_id": "CAM-0001-2026",
  "latitud": 19.5438,
  "longitud": -96.9102,
  "velocidad": 42.5,
  "registrado_en": "2026-05-24T10:30:00"
}'
```


Respuesta esperada

```
{  
  "mensaje": "Ubicación registrada correctamente.",  
  "ubicacion_id": 1  
}
```

Código HTTP esperado:

```
201 Created
```

Esta respuesta confirma que el endpoint recibió, validó y guardó la ubicación.

Paso 14. Probar errores

Prueba	Cambio realizado	Resultado esperado
Sin token	Quitar <code>Authorization</code>	<code>401</code>
Token incorrecto	Cambiar token	<code>401</code>
JSON inválido	Romper estructura JSON	<code>400</code>
Sin <code>camion_id</code>	Quitar campo	<code>400</code>
Latitud inválida	Usar <code>120</code>	<code>400</code>
Longitud inválida	Usar <code>-220</code>	<code>400</code>
Velocidad negativa	Usar <code>-10</code>	<code>400</code>

Script Python para simular camión

```
# simulador_camion.py

import requests

url = "http://127.0.0.1:8000/api/camiones/ubicacion/"

headers = {
    "Content-Type": "application/json",
    "Authorization": "Bearer TOKEN_DE_PRACTICA_CAMIONES",
}

datos = {
    "camion_id": "CAM-0001-2026",
    "latitud": 19.5438,
    "longitud": -96.9102,
    "velocidad": 42.5,
    "registrado_en": "2026-05-24T10:30:00",
}

respuesta = requests.post(url, json=datos, headers=headers)

print("Código:", respuesta.status_code)
print("Respuesta:", respuesta.json())
```

Qué representa el script

El script no es parte de Django.

Representa al camión, dispositivo o sistema externo que envía ubicaciones hacia la aplicación logística.

```
simulador_camion.py
    |
    | POST + JSON + token
    v
Django / api/camiones/ubicacion/
```

Paso 15. Verificar registros

Desde Django shell:

```
python manage.py shell
```

```
from apps.rastreo.models import UbicacionCamion

UbicacionCamion.objects.all()

for ubicacion in UbicacionCamion.objects.all():
    print(
        ubicacion.camion_id,
        ubicacion.latitud,
        ubicacion.longitud,
        ubicacion.registrado_en
    )
```

Respuestas esperadas según el caso

Caso	Evidencia esperada
Solicitud válida	Respuesta 201 y registro en base de datos
Sin token	Respuesta 401
Token incorrecto	Respuesta 401
JSON inválido	Respuesta 400
Coordenada inválida	Respuesta 400
Velocidad negativa	Respuesta 400
Consulta del historial	Registros visibles en shell o admin

Discusión de seguridad

El token estático permite simplificar la práctica, pero no es suficiente para producción.

Los estudiantes deben inferir riesgos como:

- reutilización del token;
- envío de ubicaciones falsas;
- exposición del token en código;
- ausencia de expiración;
- falta de protección contra repetición de solicitudes.

Gracias
¿Preguntas?

